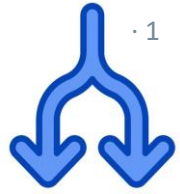
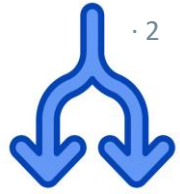


Practical Concurrent and Parallel Programming I

Intro to Concurrency and Mutual Exclusion

Raúl Pardo

Agenda



- Course General Info
- Introduction to Concurrency
- Java Threads
- Mutual Exclusion
- Java Locks

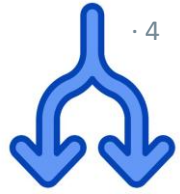
Teachers

- Course manager: Raúl Pardo
  - Associate Professor at ITU; Teaching PCPP since 2021 (and concurrency since 2013)
  - Research interest: Privacy & Security, Formal Verification, Probabilistic Programming, Concurrency

- Co-teacher: Peter Sestoft
  - Professor at ITU; Created PCPP in 2014

- Teaching Assistants
  - Abdulrahman Mohsen
  - Aiting Lee
  - Andreas Severin Hauch Trøstrup
  - Magnus Langkilde
  - Markus Kildebæk Raun Johansen
  - Rasmus Solvang
  - Viktor Horváth

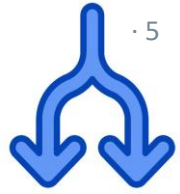
Standard weekly plan



- Lectures
  - Mondays 8-10 (recall academic quarter 8:15)
  - We publish the readings for the lecture a week before the lecture (approx.)
  - Auditorium 1

- Exercise sessions
  - Mondays 10-12 (recall academic quarter 10:15)
  - Every week we publish a set of exercises covering the material of the lecture
  - 2A12-14 (1st priority), 2A52 (2nd priority), 2A54 (if no space in others)

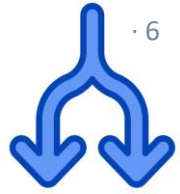
Course Info Online



- The learnIT website (https://learnit.itu.dk/course/view.php?id=3025971) contains information about the course and submission links for the assignments

- The material of the course is hosted in our github repository (https://github.itu.dk/PCPP/PCPP2026-Public)
  - Lecture slides
  - Readings
  - Example code
  - Exercises
  - Accompanying code for the exercises

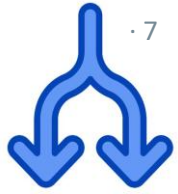
Exercises & Assignments



- You are expected to work on **groups of 2/3 people**
  - Today in the exercise session we will start by forming groups with the help of TAs
- Assignments submission deadlines are **on Mondays before the lecture**
  - We made sure to minimize clashing with other courses in the K-CS
  - In total there are 13 exercise sets, distributed in 6 assignments
- Exercises are divided into *mandatory* and *challenging*
  - Mandatory assignments are required to pass assignments
  - Challenging exercises are meant for students aiming at high grades in the exam
  - Challenging exercises are not required to pass the assignments
- Assignments submission:
  - In LearnIT, you submit link to a repository in github.itu.dk
  - Repositories should have only one branch
  - The repository should be public
  - A member of the group submits on behalf of all members of the group (but grades are individual)

You can find detailed info about assignment submissions and feedback in the GitHub repo

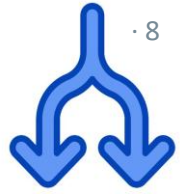
Exercises Guides



- We use the build tool Gradle to compile and run Java code for exercises
- There is a guide in the course repo with instructions on how to install it and run exercises code

- The last part of the course uses the programming language Erlang
- Erlang is a functional language
- There is a guide to get started with exercises
- **If you do not know Erlang, we recommend that you start learning from today**
  - The guide contains a link to a textbook to get started with Erlang

Oral feedback & Assessment



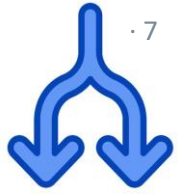
- Feedback and assessment of assignments is provided in oral sessions

- You **must book an oral feedback slot with a TA/teacher** using the organizer in learnIT
  - Today in the exercise session you can already book a slot (https://learnit.itu.dk/mod/organizer/view.php?id=250667)

- You must **pass 5 assignments** to be entitled **to take the exam**
  - We encourage you to not skip assignments, as they are one of the best activities to prepare for the exam

You can find detailed info about assignment submissions and feedback in the GitHub repo

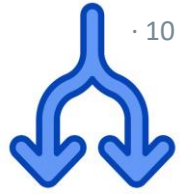
Questions and Answers Forum



- The main channel of communication is the Questions and Answers Forum in LearnIT
  - https://learnit.itu.dk/mod/hsuforum/view.php?id=250665
    We strongly encourage you to use the forum!
  - We will check the forum regularly
  - But we also encourage you to answers questions yourself!
    - The forum is meant to be a discussion platform to boost learning and share knowledge

- The only constraint: *do not directly post solutions to exercises*

Exam



- Oral exam
  - 30 min (including grade deliberation)
  - Questions about any part of the syllabus

- Oral feedback sessions aim at preparing you for the exam

GenAI policy

- You can use GenAI
  - Use it responsibly; as a learning tool
  - Follow the official ITU Guidelines: https://itustudent.itu.dk/Study-Administration/Check-this-out/Generative-AI

- During the examination GenAI is not allowed

Blue box questions

- Short questions to discuss on the spot during the lecture
  - Please try to answer them!

What is this blue box?

- Blue questions method: (*pair-)think-ink-share*:
  - Think about the question for (2-3 min)
  - Write the answer in mentimeter
    (or on a piece of paper, or in your laptop, …) (1 min)
  - Then, we pose the question in class, and you may share your answer

menti.com
6643 7213

Concurrency

It takes one person 2 hours to dig 4 meters of ditch.
How long will it take 2 people to dig 4 meters of ditch?

Concurrency

It takes one person 2 hours to dig 4 meters of ditch.
How long will it take 2 people to dig 4 meters of ditch?

It takes your PC 2 minutes to add 4 billion numbers.
How long will it take 2 PCs to add 4 billion numbers?

Concurrency

It takes one person 2 hours to dig 4 meters of ditch.
How long will it take 2 people to dig 4 meters of ditch?

What if they only have one shovel?

It takes your PC 2 minutes to add 4 billion numbers.
How long will it take 2 PCs to add 4 billion numbers?

Concurrency

It takes one person 2 hours to dig 4 meters of ditch.
How long will it take 2 people to dig 4 meters of ditch?

What if they only have one shovel?

It takes your PC 2 minutes to add 4 billion numbers.
How long will it take 2 PCs to add 4 billion numbers?

What if they must read from the same memory?

Concurrency

It takes one person 2 hours to dig 4 meters of ditch.
How long will it take 2 people to dig 4 meters of ditch?

What if they only have one shovel?

What if they must dig a hole (and not a ditch)?

It takes your PC 2 minutes to add 4 billion numbers.
How long will it take 2 PCs to add 4 billion numbers?

What if they must read from the same memory?

Concurrency

It takes one person 2 hours to dig 4 meters of ditch.
How long will it take 2 people to dig 4 meters of ditch?

What if they only have one shovel?

What if they must dig a hole (and not a ditch)?

It takes your PC 2 minutes to add 4 billion numbers.
How long will it take 2 PCs to add 4 billion numbers?

What if they must read from the same memory?

What if they must sort (and not add)?

Concurrency

It takes one person 2 hours to dig 4 meters of ditch.
How long will it take 2 people to dig 4 meters of ditch?

What if they only have one shovel?

What if they must dig a hole (and not a ditch)?

How fast can a 100 persons dig 4 meters of ditch?

It takes your PC 2 minutes to add 4 billion numbers.
How long will it take 2 PCs to add 4 billion numbers?

What if they must read from the same memory?

What if they must sort (and not add)?

Concurrency

It takes one person 2 hours to dig 4 meters of ditch.
How long will it take 2 people to dig 4 meters of ditch?

What if they only have one shovel?

What if they must dig a hole (and not a ditch)?

How fast can a 100 persons dig 4 meters of ditch?

It takes your PC 2 minutes to add 4 billion numbers.
How long will it take 2 PCs to add 4 billion numbers?

What if they must read from the same memory?

What if they must sort (and not add)?

How fast can a 100 PCs sort 4 billion numbers?

Concurrency

It takes one person 2 hours to dig 4 meters of ditch.
How long will it take 2 people to dig 4 meters of ditch?

What if they only have one shovel?

What if they must dig a hole (and not a ditch)?

How fast can a 100 persons dig 4 meters of ditch?

…

It takes your PC 2 minutes to add 4 billion numbers.
How long will it take 2 PCs to add 4 billion numbers?

What if they must read from the same memory?

What if they must sort (and not add)?

How fast can a 100 PCs sort 4 billion numbers?

…

Concurrency is everywhere!

Almost any computer system nowadays uses concurrency

- Any web application
- Any User Interface (UI)
- Operating systems
- Search engines
- Messaging applications
- Audio/Video Streaming
- GPS navigation
- Machine learning, LLMs
- …

Concurrency is everywhere!

Almost any computer system nowadays uses concurrency

- Any web application
- Any User Interface (UI)
- Operating systems
- Search engines
- Messaging applications
- Audio/Video Streaming
- GPS navigation
- Machine learning, LLMs
- …

In this course, you will learn the fundamental concurrency principles that these systems use

You will learn multiple methods to develop concurrent software, **analyze its correctness and evaluate its performance**

Concurrency

Doing more than one computation at a time

Concurrency

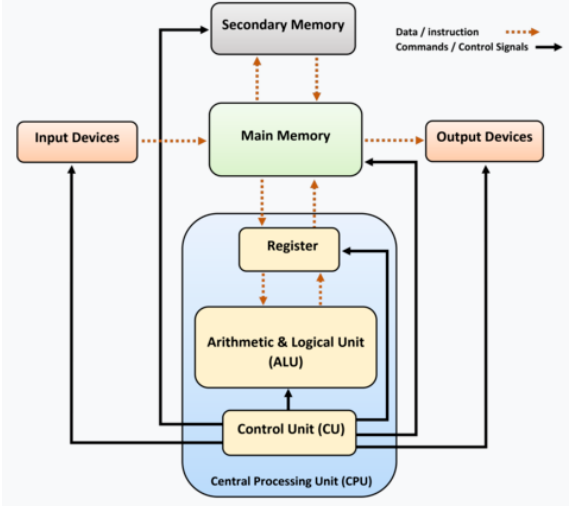
Doing more than one computation at a time

Hardware matters !

Concurrency

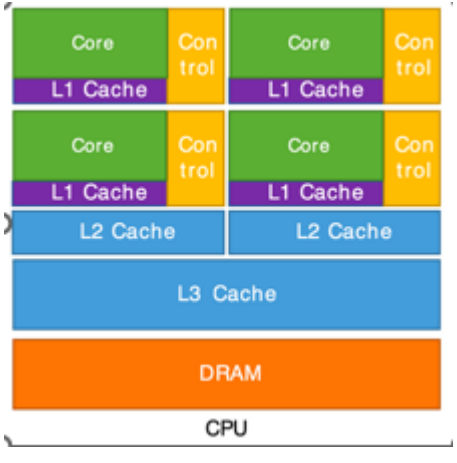
Doing more than one computation at a time

Hardware matters !

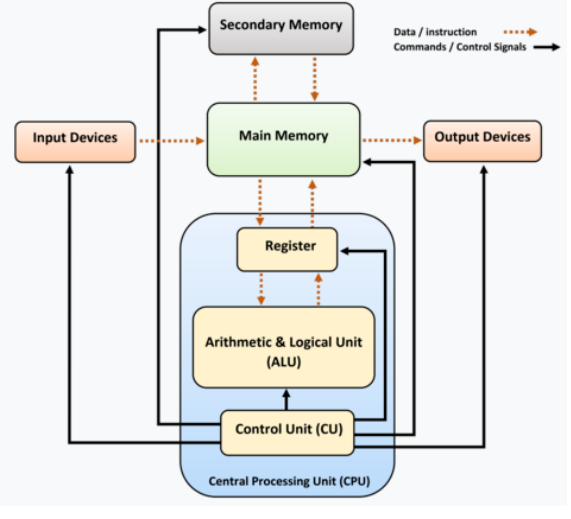


Concurrency

Doing more than one computation at a time



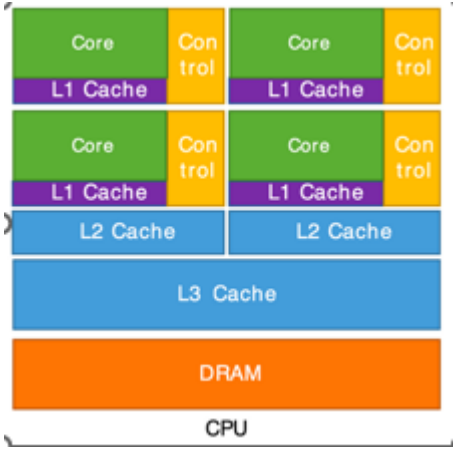
Hardware matters !

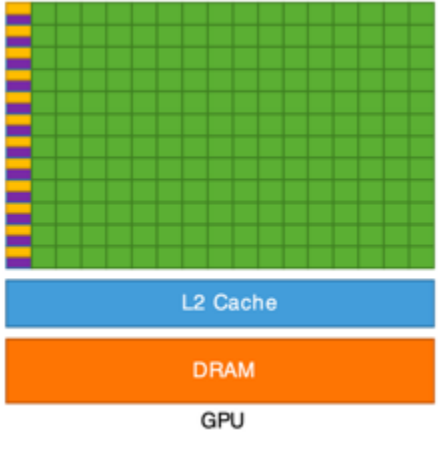


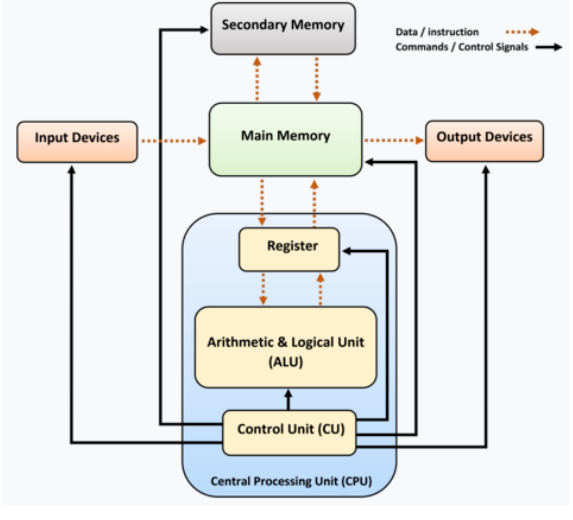
Concurrency

Doing more than one computation at a time

Hardware matters !



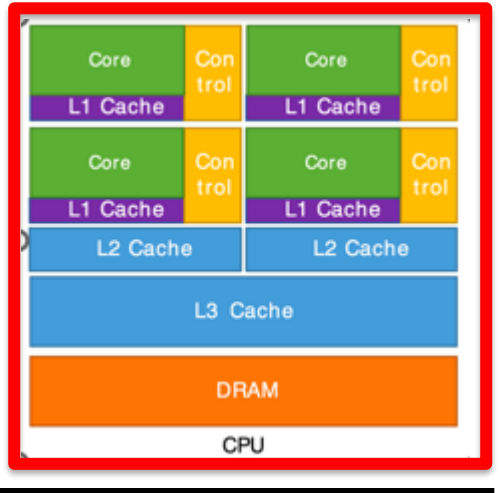


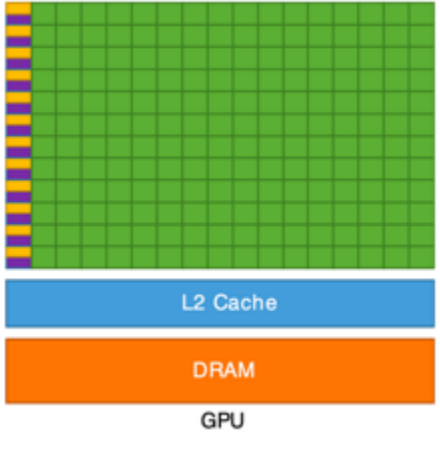


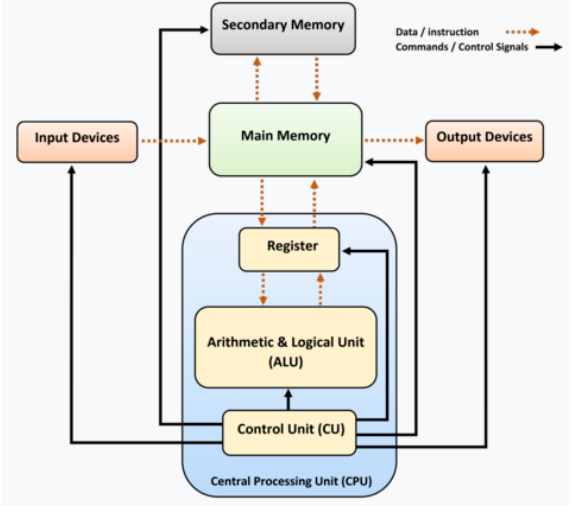
Concurrency

Doing more than one computation at a time

Hardware matters !







Sequential vs Parallel vs Concurrent

- **Sequential** execution (in 1 processing core)

Sequential vs Parallel vs Concurrent

- **Sequential** execution (in 1 processing core)

- **Parallel** execution (in 2 processing cores)

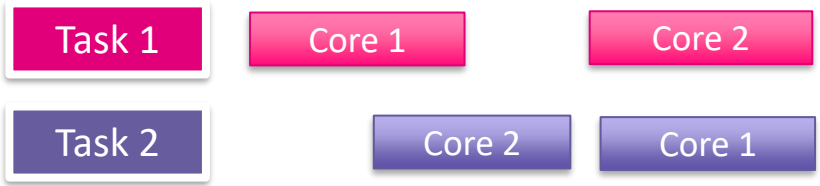
Sequential vs Parallel vs Concurrent

- **Sequential** execution (in 1 processing core)

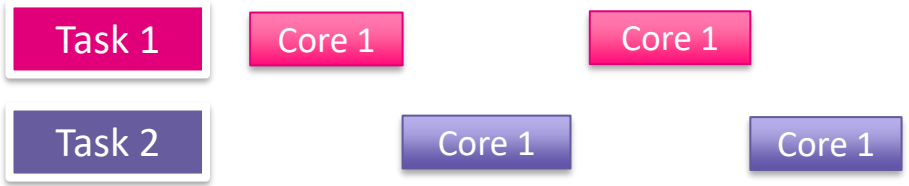
- **Parallel** execution (in 2 processing cores)

- **Concurrent** execution

Multiple processing cores



One processing core



Sequential vs Parallel vs Concurrent

- **Sequential** execution (in 1 processing core)

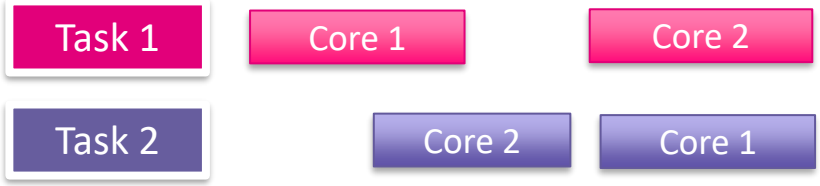
- **Parallel** execution (in 2 processing cores)

Concurrent

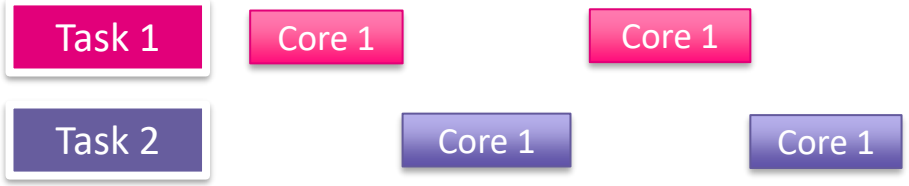
- Computation may be split in arbitrary blocks (as opposed to sequential)
- Multiple concurrent streams can be executed in one core (as opposed to parallel)
- When there are more than two cores different threads may run in parallel

- **Concurrent** execution

Multiple processing cores



One processing core



Single vs multiple streams

Single vs multiple streams

```
public class RemoveDuplicateInArrayExample{
  public static int removeDuplicateElements(int arr[], int n){
    if (n==0 || n==1){  return n; }
    int[] temp = new int[n];
    int j = 0;
    for (int i=0; i<n-1; i++){
      if (arr[i] != arr[i+1]){
        temp[j++] = arr[i];
      }
     }
    temp[j++] = arr[n-1];
    // Changing original array
    for (int i=0; i<j; i++){
      arr[i] = temp[i];
    }
    return j;
  }

  public static void main (String[] args) {
    int arr[] = {10,20,20,30,30,40,50,50};
    int length = arr.length;
    length = removeDuplicateElements(arr, length);
    //printing array elements
    for (int i=0; i<length; i++)   System.out.print(arr[i]+" ");
  }
}
```

Single stream

```
----------
----
--------
-----
---
-------
```

Single vs multiple streams

```
public class RemoveDuplicateInArrayExample{
  public static int removeDuplicateElements(int arr[], int n){
    if (n==0 || n==1){  return n; }
    int[] temp = new int[n];
    int j = 0;
    for (int i=0; i<n-1; i++){
      if (arr[i] != arr[i+1]){
        temp[j++] = arr[i];
      }
     }
    temp[j++] = arr[n-1];
    // Changing original array
    for (int i=0; i<j; i++){
      arr[i] = temp[i];
    }
    return j;
  }

  public static void main (String[] args) {
    int arr[] = {10,20,20,30,30,40,50,50};
    int length = arr.length;
    length = removeDuplicateElements(arr, length);
    //printing array elements
    for (int i=0; i<length; i++)   System.out.print(arr[i]+" ");
  }
}
```

Single stream

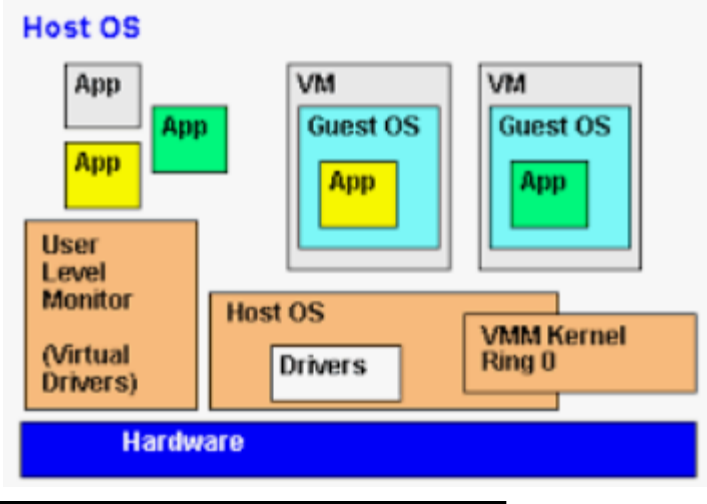
```
------------
----
--------
-----
---
-------
```

Concurrent (multiple streams)

```
----------    ----    ------------
----          ---------    ----
--------      -----    --------
-----         --------    -----
---           ---------    ---
-------       --------    -------
              ---------
              -----
```

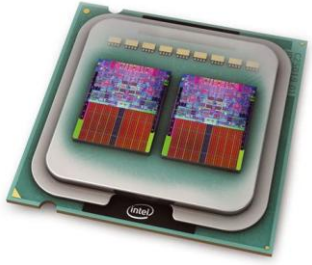
Motivations for Concurrency

Hidden (virtual)

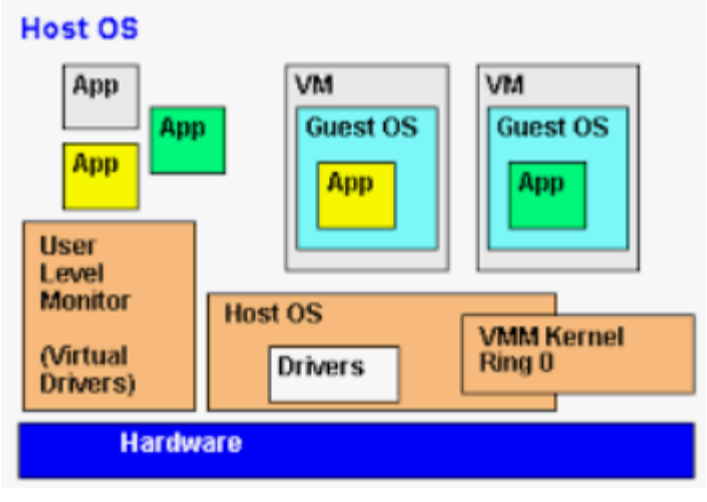


Motivations for Concurrency

Exploitation

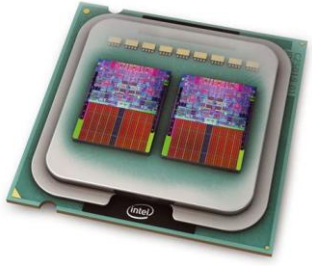


Hidden (virtual)



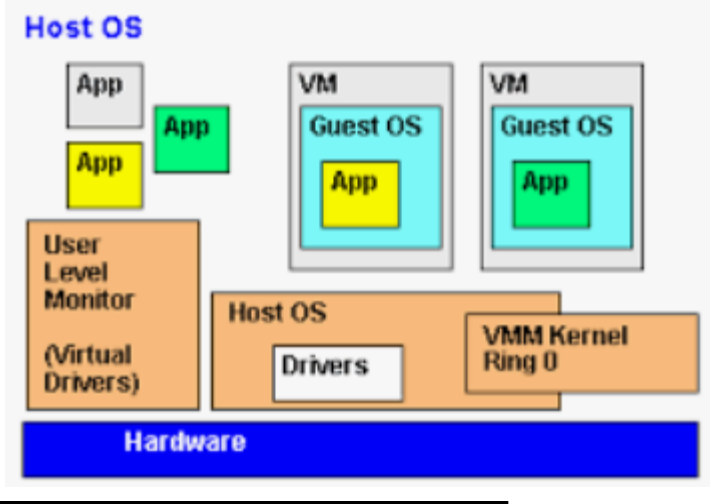
Motivations for Concurrency

Exploitation



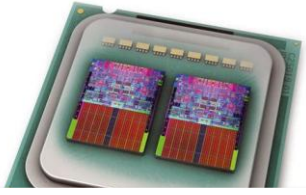
Inherent

Hidden (virtual)



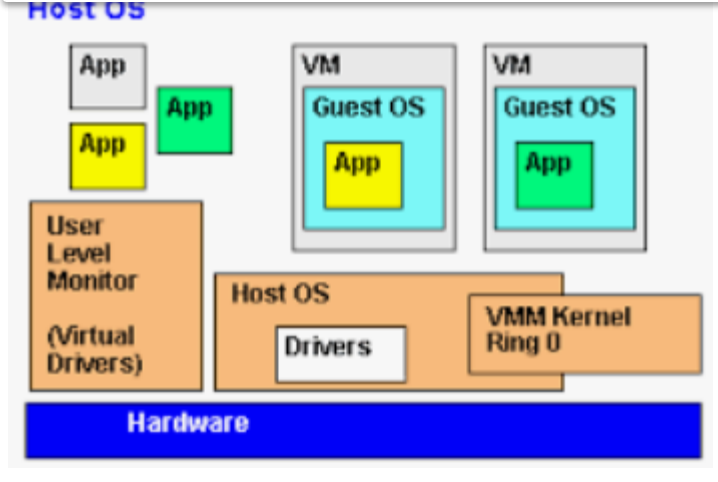
Motivations for Concurrency

Exploitation



Inherent

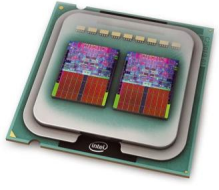
Concurrency is an abstraction for all of these (and more)



Concurrency

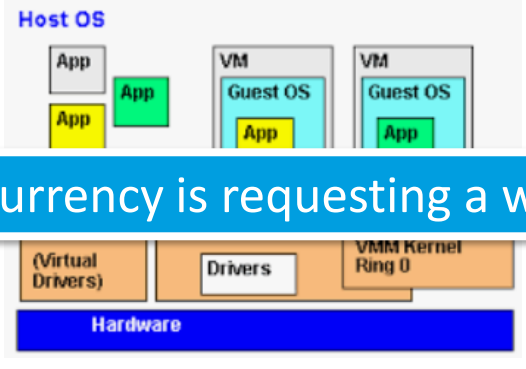
Concepts, challenges, theories that are common to:

Exploitation



Inherent

Hidden (virtual)



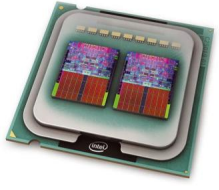
What kind of concurrency is requesting a web-page from a server?

Concurrency

Concepts, challenges, theories that are common to:

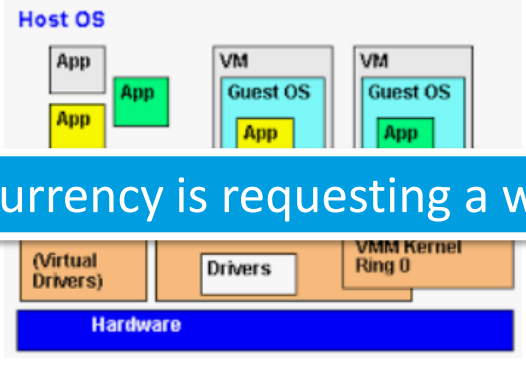
menti.com
6643 7213

Exploitation



Inherent

Hidden (virtual)



What kind of concurrency is requesting a web-page from a server?

Programming for concurrency

In PCPP we will study (in detail) the mechanisms/abstractions to program for concurrency in Java (and in less detail a few other languages).

Programming for concurrency

In PCPP we will study (in detail) the mechanisms/abstractions to program for concurrency in Java (and in less detail a few other languages).

It is important to *distinguish* between the *abstract concept concurrency* and the *language/hardware details* used to implement concurrency!

Programming for concurrency

In PCPP we will study (in detail) the mechanisms/abstractions to program for concurrency in Java (and in less detail a few other languages).

It is important to *distinguish* between the *abstract concept concurrency* and the *language/hardware details* used to implement concurrency!

Abstract: **streams:** s1;s2;s3; ….
z1;z2;z3;z4; …
…..

Programming for concurrency

In PCPP we will study (in detail) the mechanisms/abstractions to program for concurrency in Java (and in less detail a few other languages).

It is important to *distinguish* between the *abstract concept concurrency* and the *language/hardware details* used to implement concurrency!

Abstract: **streams:** s1;s2;s3; ….
z1;z2;z3;z4; …
…..

+

cordination: s1;s2;s3; ….

z1;z2;z3;z4; …
…..

Programming for concurrency

In PCPP we will study (in detail) the mechanisms/abstractions to program for concurrency in Java (and in less detail a few other languages).

It is important to *distinguish* between the *abstract concept concurrency* and the *language/hardware details* used to implement concurrency!

Abstract: **streams:** s1;s2;s3; ….
z1;z2;z3;z4; …
…..

+

cordination: s1;s2;s3; ….

z1;z2;z3;z4; …
…..

Concrete: Java threads / callbacks / processes / tasks / coroutines …

Programming for concurrency

In PCPP we will study (in detail) the mechanisms/abstractions to program for concurrency in Java (and in less detail a few other languages).

It is important to *distinguish* between the *abstract concept concurrency* and the *language/hardware details* used to implement concurrency!

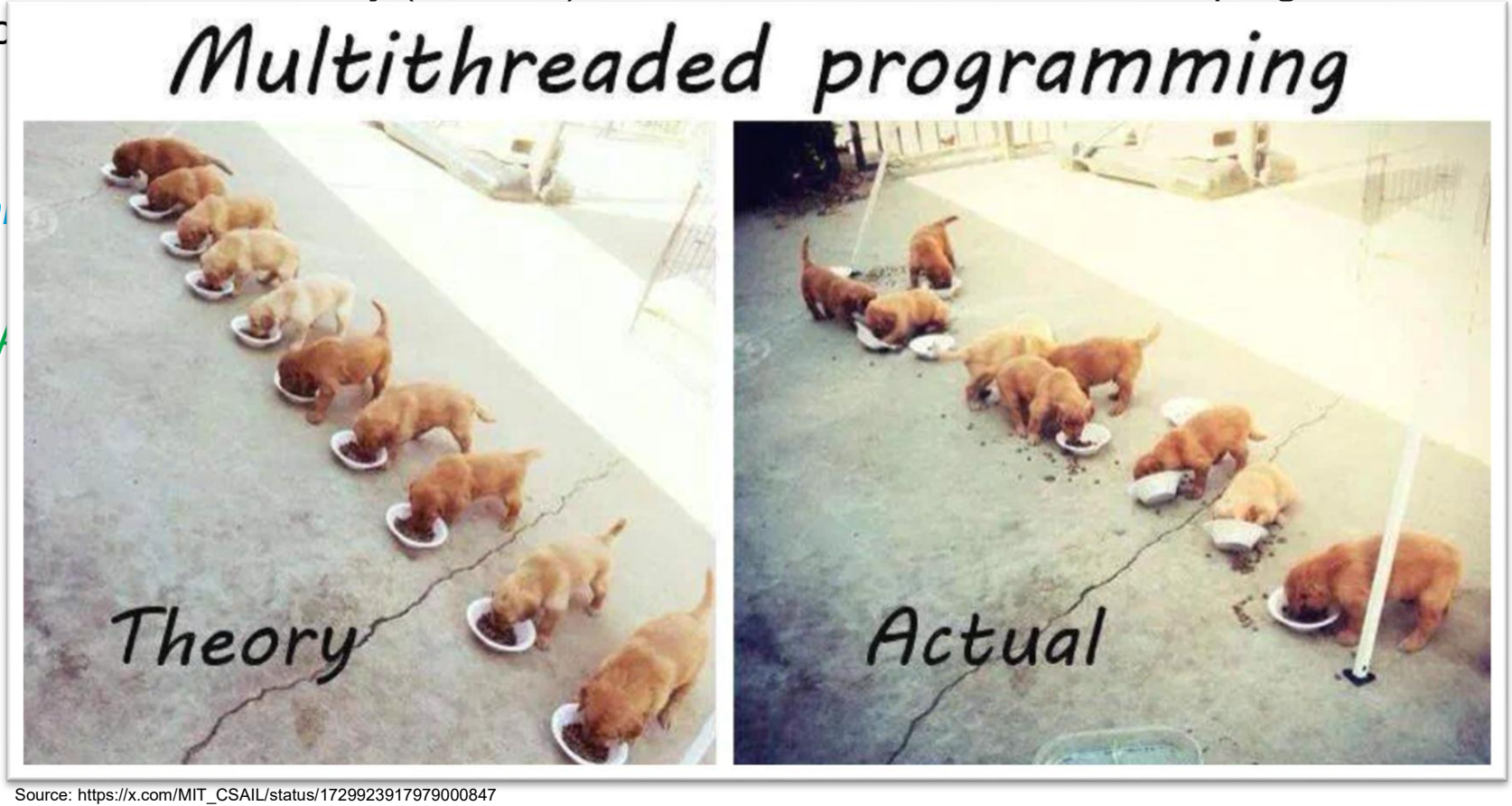
Abstract: **streams:** s1;s2;s3; ….
z1;z2;z3;z4; …
…..
+
cordination: s1;s2;s3; ….
z1;z2;z3;z4; …
…..

Concrete: Java threads / callbacks / processes / tasks / coroutines …
+
lock / monitor / semaphore / message / ...

Programming for concurrency

In PCPP we will study (in detail) the mechanisms/abstractions to program for co

It
la

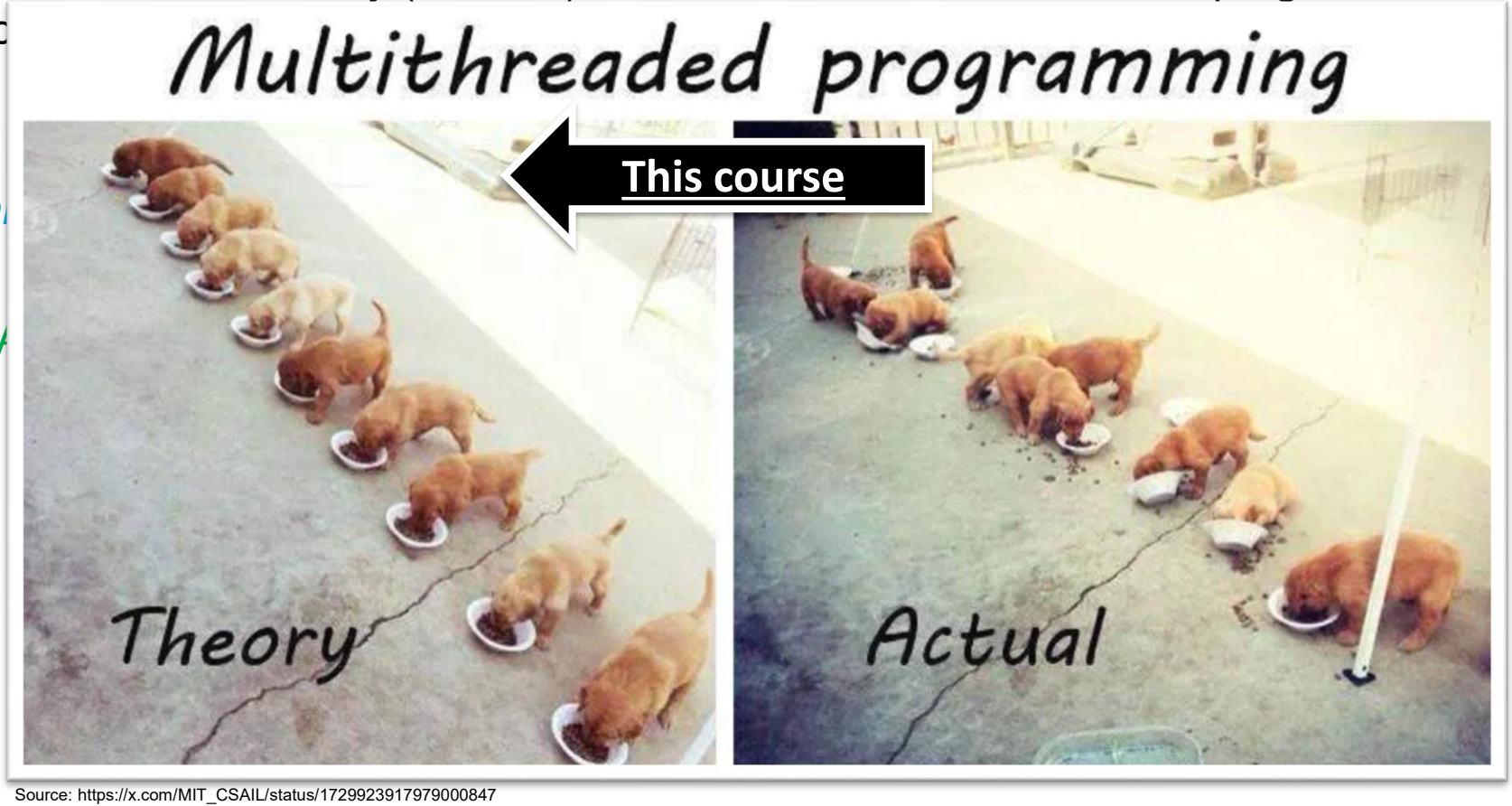


Source: https://x.com/MIT_CSAIL/status/1729923917979000847

Programming for concurrency

In PCPP we will study (in detail) the mechanisms/abstractions to program for co

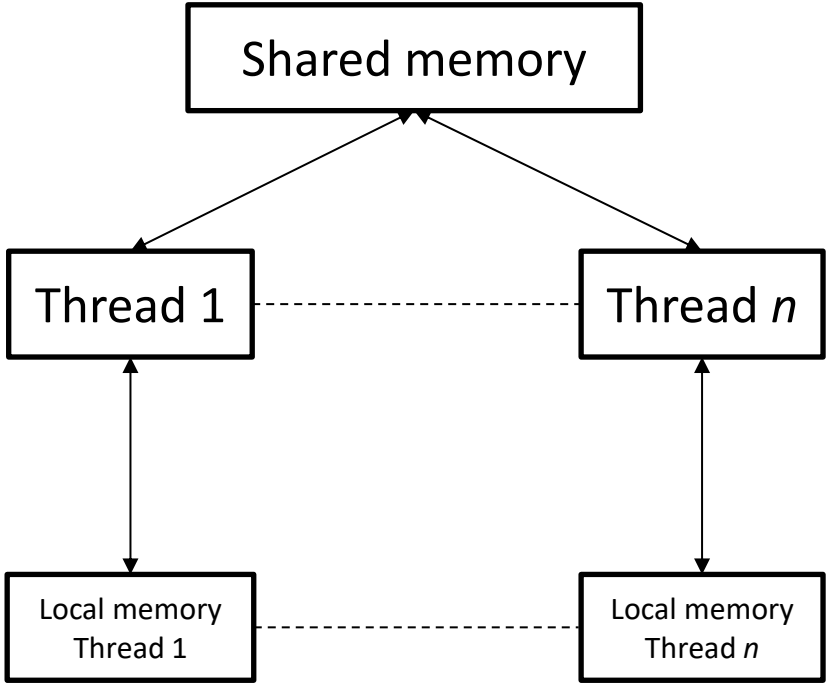
It
la



Source: https://x.com/MIT_CSAIL/status/1729923917979000847

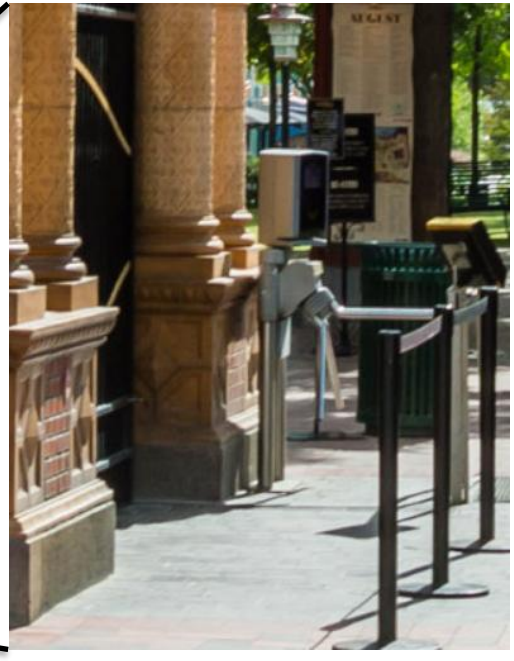
Threads in Java

- A *thread* is an independent sequence of program statements
- Several threads can be executed at the same time, i.e., *concurrently*
- Each threads works at its own speed
- Each thread has its own local memory
- Threads can communicate via shared memory



Threads in Java – Example

Tivoli entrance turnstile



Threads in Java - Example

- Java threads can be defined as classes that implement **`Runnable`** or extend from **`Thread`**
- The behaviour of the thread is in the **`run()`** method (override)

```
final long PEOPLE = 10_000;
long counter = 0;
…

public class Turnstile extends Thread {
   public void run() {
        for (int i = 0; i < PEOPLE; i++) {
            counter++;
        }
   }
}
```

This thread simulates 10000 people entering to Tivoli

Threads in Java - Example

- Java threads can be defined as classes that implement **`Runnable`** or extend from **`Thread`**
- The behaviour of the thread is in the **`run()`** method (override)

```
final long PEOPLE = 10_000;
long counter = 0;
…

public class Turnstile extends Thread {
   public void run() {
        for (int i = 0; i < PEOPLE; i++) {
            counter++;
        }
   }
}
```

Shared memory

This thread simulates 10000 people entering to Tivoli

Threads in Java - Example

- Java threads can be defined as classes that implement **`Runnable`** or extend from **`Thread`**
- The behaviour of the thread is in the **`run()`** method (override)

```
final long PEOPLE = 10_000;
long counter = 0;
...

public class Turnsti                Thread {
   public void run()
        for (int i = 0; i < PEOPLE; i++) {
            counter++;
        }
   }
}
```

This thread simulates 10000 people entering to Tivoli

Shared memory

Local memory

Threads in Java - Example

- Java threads can be defined as classes that implement **`Runnable`** or extend from **`Thread`**
- The behaviour of the thread is in the **`run()`** method (override)

```
final long PEOPLE = 10_000;
long counter = 0;
...

public class Turnsti                Thread {
   public void run()
        for (int i = 0; i < PEOPLE; i++) {
            counter++;
        }
   }
}
```

This thread simulates 10000 people entering to Tivoli

Shared memory

Local memory

Behaviour of the thread

Threads in Java - Example

- The function **`start()`** starts the execution of the thread
- The function **`join()`** waits for the thread to terminate

```
Turnstile turnstile = new Turnstile();

turnstile.start();

turnstile.join();

System.out.println(counter+" people entered");
```

Create an instance of the thread

Start execution of the thread

Wait until the thread terminates

Print the value of **`counter`**

Threads in Java - Example

- Altogether (not executable, see **`CounterThreads.java`** for the executable program)

```
class CounterThreads {
    long counter = 0;
    final long PEOPLE  = 10_000;


    // main thread behaviour
    Turnstile turnstile = new Turnstile();
    turnstile.start();
    turnstile.join();
    System.out.println(counter+" people entered");


    // inner class for accessing shared variables
    public class Turnstile extends Thread {
       public void run() {
           for (int i = 0; i < PEOPLE; i++) {
                counter++;
           }
       }
    }
}
```

Threads in Java - Example

- Altogether (not executable, see **`CounterThreads.java`** for the executable program)

```
class CounterThreads {
    long counter = 0;
    final long PEOPLE  = 10_000;


    // main thread behaviour
    Turnstile turnstile = new Turnstile();
    turnstile.start();
    turnstile.join();
    System.out.println(counter+" people entered");


    // inner class for accessing shared variables
    public class Turnstile extends Thread {
       public void run() {
           for (int i = 0; i < PEOPLE; i++) {
                counter++;
           }
       }
    }
}
```

Shared memory

Create, start, wait till termination and print results

Definition of the thread´s behaviour

Threads in Java - Example

What value of **`counter`** will this program print?

ecutable, see **`CounterThreads.java`** for
gram)

Shared memory

Create, start, wait till termination and print results

Definition of the thread´s behaviour

```
class CounterThreads {
    long counter = 0;
    final long PEOPLE  = 10_000;


    // main thread behaviour
    Turnstile turnstile = new Turnstile();
    turnstile.start();
    turnstile.join();
    System.out.println(counter+" people entered");


    // inner class for accessing shared variables
    public class Turnstile extends Thread {
       public void run() {
           for (int i = 0; i < PEOPLE; i++) {
                counter++;
           }
       }
    }
}
```

Threads in Java – Alternative Definition

Other ways to define threads

Runnable object in the thread constructor

```
long counter = 0;
final long PEOPLE  = 10_000;


Thread t = new Thread(new Runnable(){
    public void run() {
           for (int i=0; i<PEOPLE; i++){
                counter++;
           }
    }
});
t.start();
t.join();
System.out.println(counter+" people entered");
```

Using Java lambda expressions

```
long counter = 0;
final long PEOPLE  = 10_000;


Thread t = new Thread(() -> {
    for (int i=0; i<PEOPLE; i++){
        counter++;
    }
});
t.start();
t.join();
System.out.println(counter+" people entered");
```

Threads in Java – Alternative Definition

Other ways to define threads

Runnable object in the thread constructor

```
long counter = 0;
final long PEOPLE  = 10_000;


Thread t = new Thread(new Runnable(){
    public void run() {
           for (int i=0; i<PEOPLE; i++){
                counter++;
           }
    }
});
t.start();
t.join();
System.out.println(counter+" people entered");
```

Using Java lambda expressions

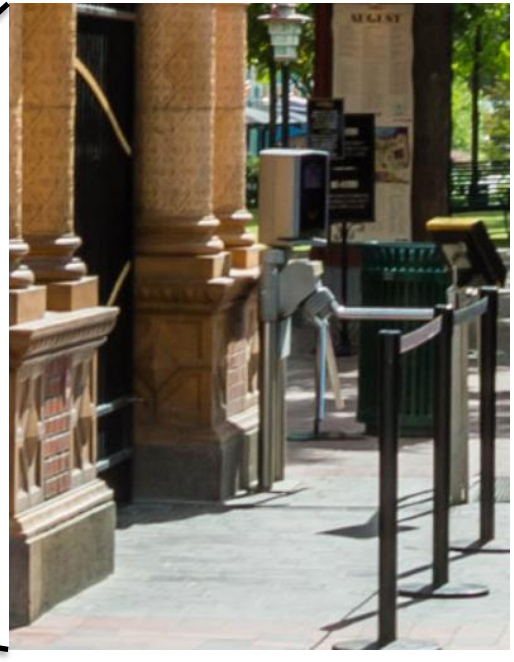
```
long counter = 0;
final long PEOPLE  = 10_000;


Thread t = new Thread(() -> {
    for (int i=0; i<PEOPLE; i++){
        counter++;
    }
});
t.start();
t.join();
System.out.println(counter+" people entered");
```

I would only recommend these when the thread´s code is small, e.g., without several methods. And when the local state of the thread is minimal.

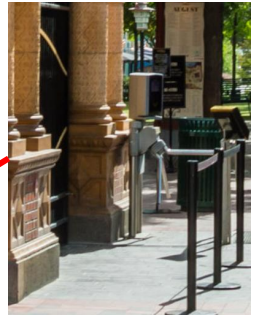
Threads in Java – Example

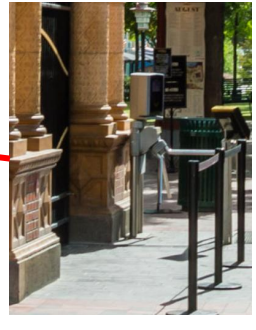
Tivoli entrance turnstile



Threads in Java – Example II

Tivoli entrance turnstile





Threads in Java – Example II

- Altogether (not executable, see **`CounterThreads2.java`** for the executable program)

```
long counter = 0;
final long PEOPLE  = 10_000;


Turnstile turnstile1 = new Turnstile();
Turnstile turnstile2 = new Turnstile();
turnstile1.start();turnstile2.start();
turnstile1.join();turnstile2.join();
System.out.println(counter+" people entered");



public class Turnstile extends Thread {
    public void run() {
        for (int i = 0; i < PEOPLE; i++) {
          counter++;
        }
    }
}
```

We simply add another Turnstile thread

Threads in Java – Example II

What value of **counter** will this program print?

ecutable, see **CounterThreads2.java**

gram)

```
long counter = 0;
final long PEOPLE  = 10_000;


Turnstile turnstile1 = new Turnstile();
Turnstile turnstile2 = new Turnstile();
turnstile1.start();turnstile2.start();
turnstile1.join();turnstile2.join();
System.out.println(counter+" people entered");



public class Turnstile extends Thread {
    public void run() {
        for (int i = 0; i < PEOPLE; i++) {
          counter++;
        }
    }
}
```

We simply add another Turnstile thread

menti.com

6643 7213

Threads in Java – Example II

What value of **counter** will this program print?

ecutable, see **CounterThreads2.java**

gram)

menti.com

6643 7213

```
long counter = 0;
final long PEOPLE  = 10_000;


Turnstile turnstile1 = new Turnstile();
Turnstile turnstile2 = new Turnstile();
turnstile1.start();turnstile2.start();
turnstile1.join();turnstile2.join();
System.out.println(counter+" people entered");



public class Turnstile extends Thread {
    public void run() {
        for (int i = 0; i < PEOPLE; i++) {
          counter++;
        }
    }
}
```

We simply add another Turnstile thread

Threads in Java – Example II

What value of **`counter`** will this program print?

ecutable, see **`CounterThreads2.java`**
gram)

menti.com
6643 7213

```
long counter = 0;
final long PEOPLE  = 10_000;


Turnstile turnstile1 = new Turnstile();
Turnstile turnstile2 = new Turnstile();
turnstile1.start();turnstile2.start();
turnstile1.join();turnstile2.join();
System.out.println(counter+" people entered");



public class Turnstile extends Thread {
    public void run() {
                                          i++) {
}
```

We simply add another Turnstile thread

HARDer: What is the minimum value of **`counter`** that this program can print?

Threads in Java – Example II

- What was is the problem in the previous program?

- To answer this question, we need to understand
  - Atomicity
  - States of a thread
  - Non-determinism
  - Interleavings

Atomicity

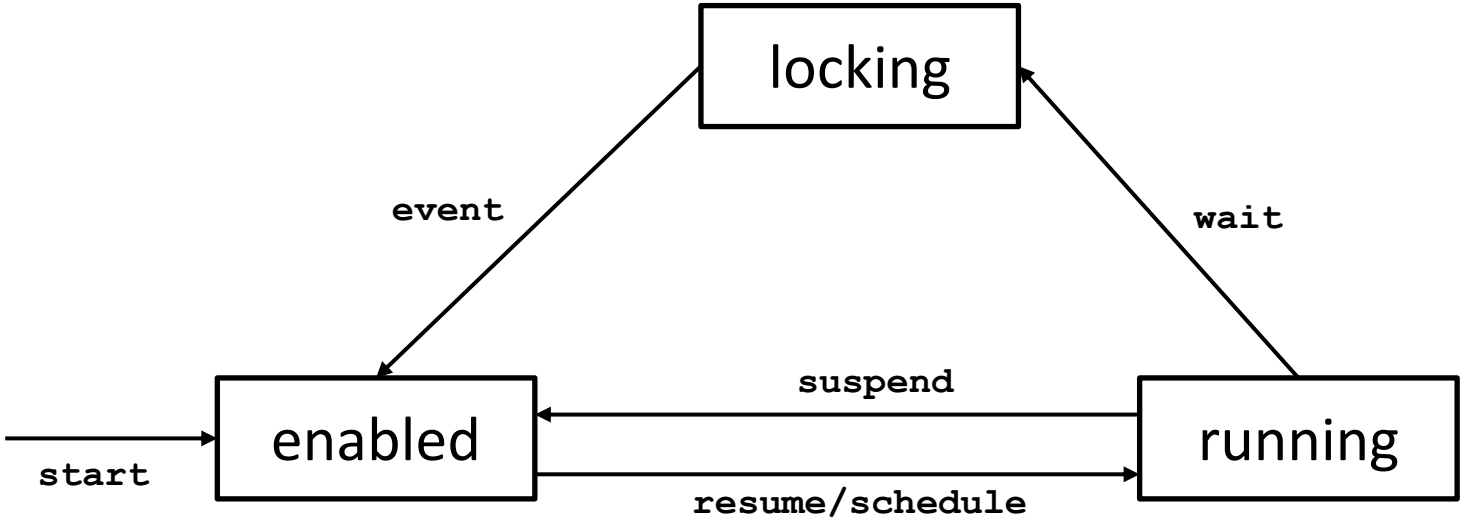
- The program statement **`counter++`** is not *atomic*
- *An atomic statement is executed as if it is a single (indivisible) operation*

```
public class Turnstile extends Thread {
   public void run() {
        for (int i = 0; i < PEOPLE; i++) {
            counter++;
        }
   }
}
```

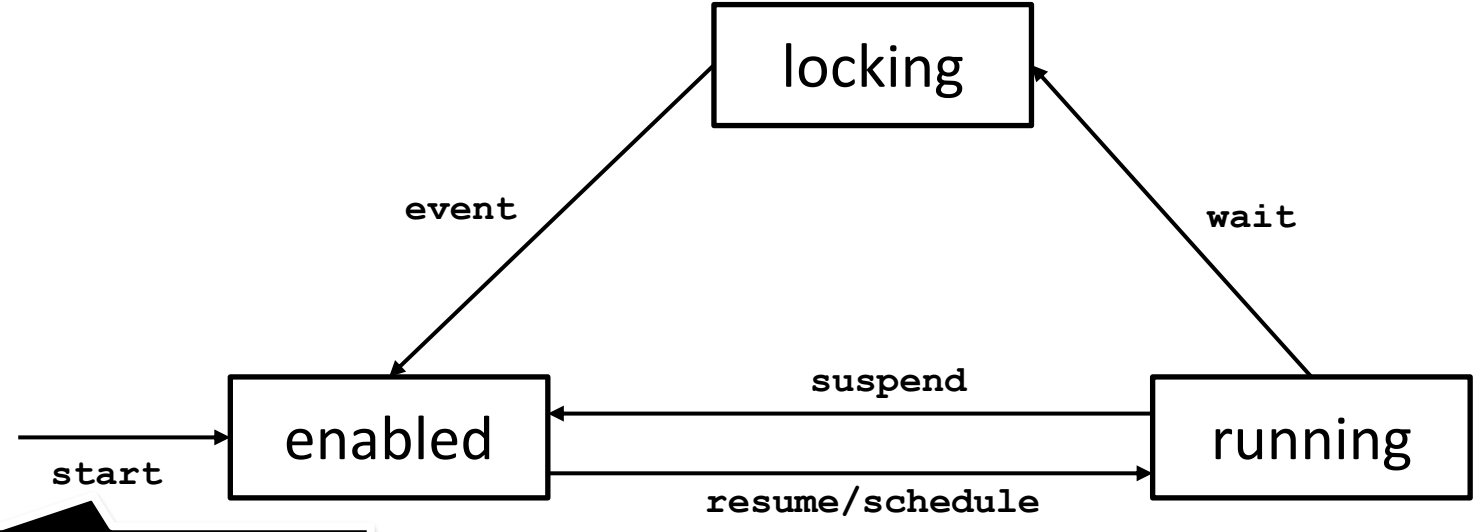
```
int temp = counter;
counter = temp + 1;
```

Watchout: Just because a program statement is a one-liner, it doesn´t mean that it is atomic

States of a thread (simplified)

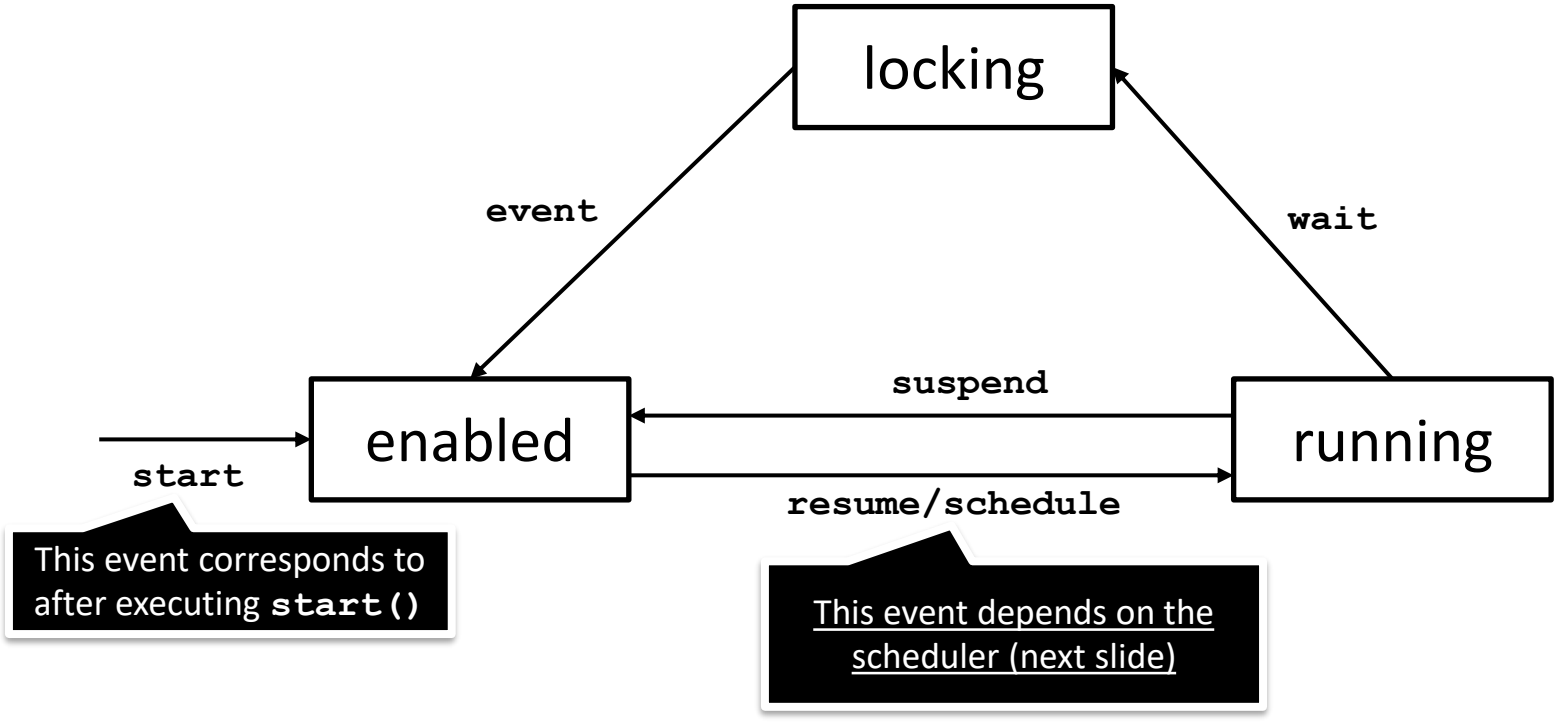


States of a thread (simplified)

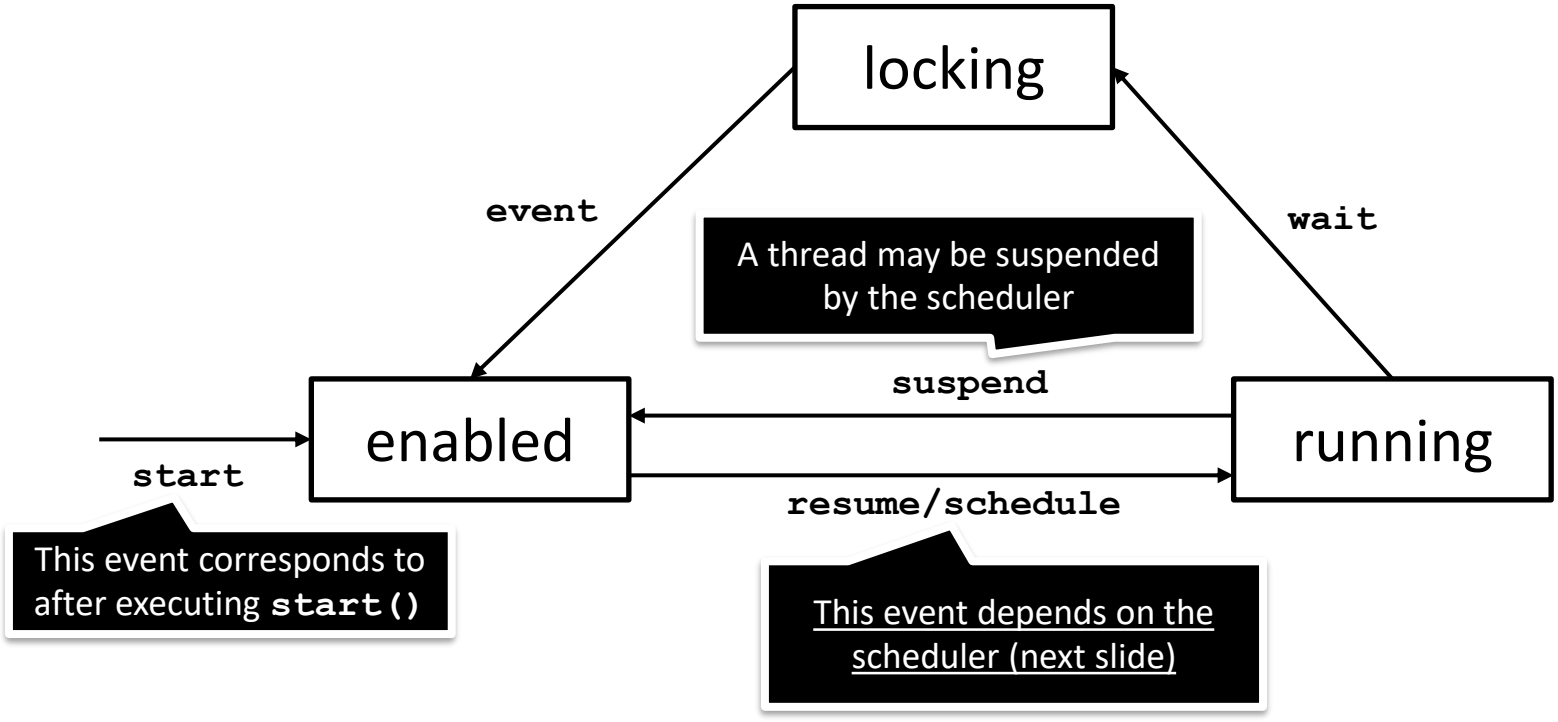


This event corresponds to after executing `start()`

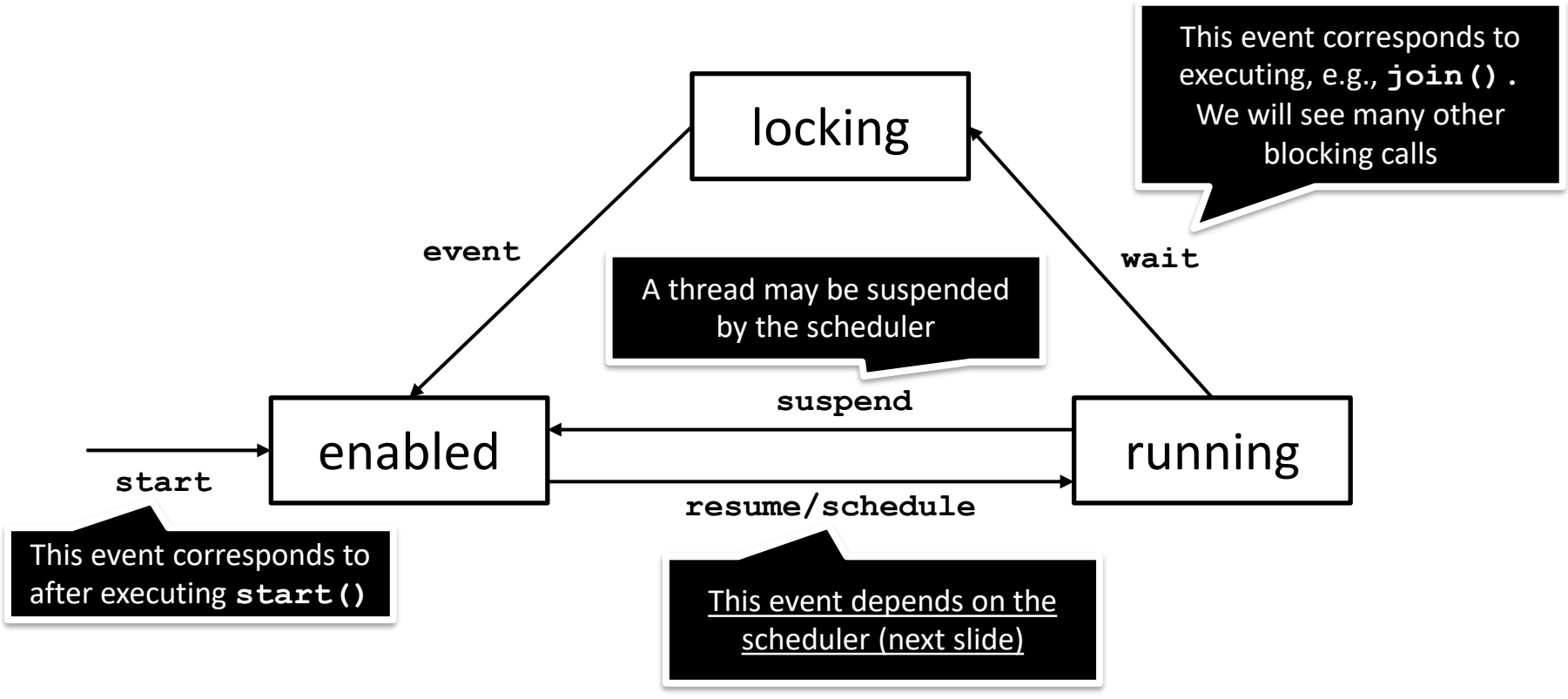
States of a thread (simplified)



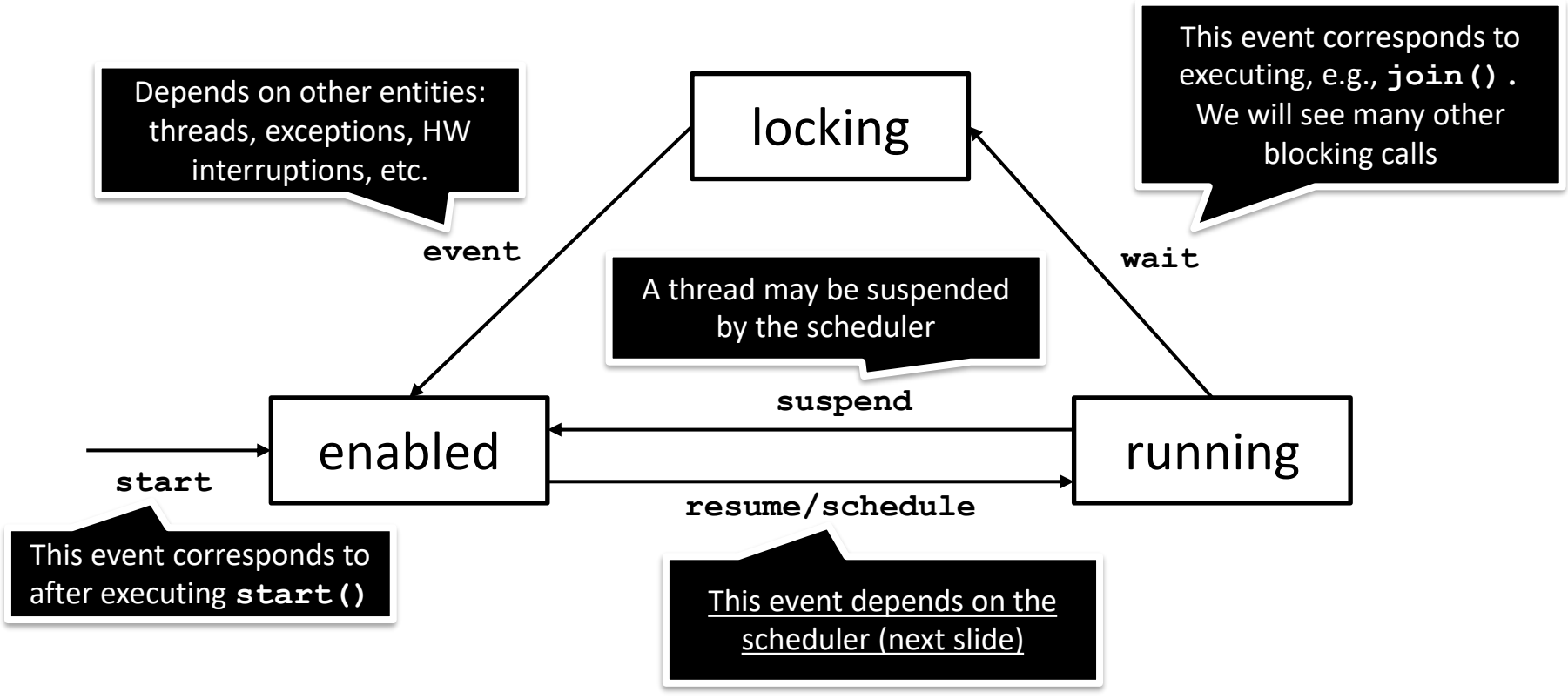
States of a thread (simplified)



States of a thread (simplified)



States of a thread (simplified)



Non-determinism | Scheduler

- In all operating systems/executing environments a *scheduler* selects the processes/threads under execution
  - Threads are selected *non-deterministically*, i.e., no assumptions can be made about what thread will be executed next

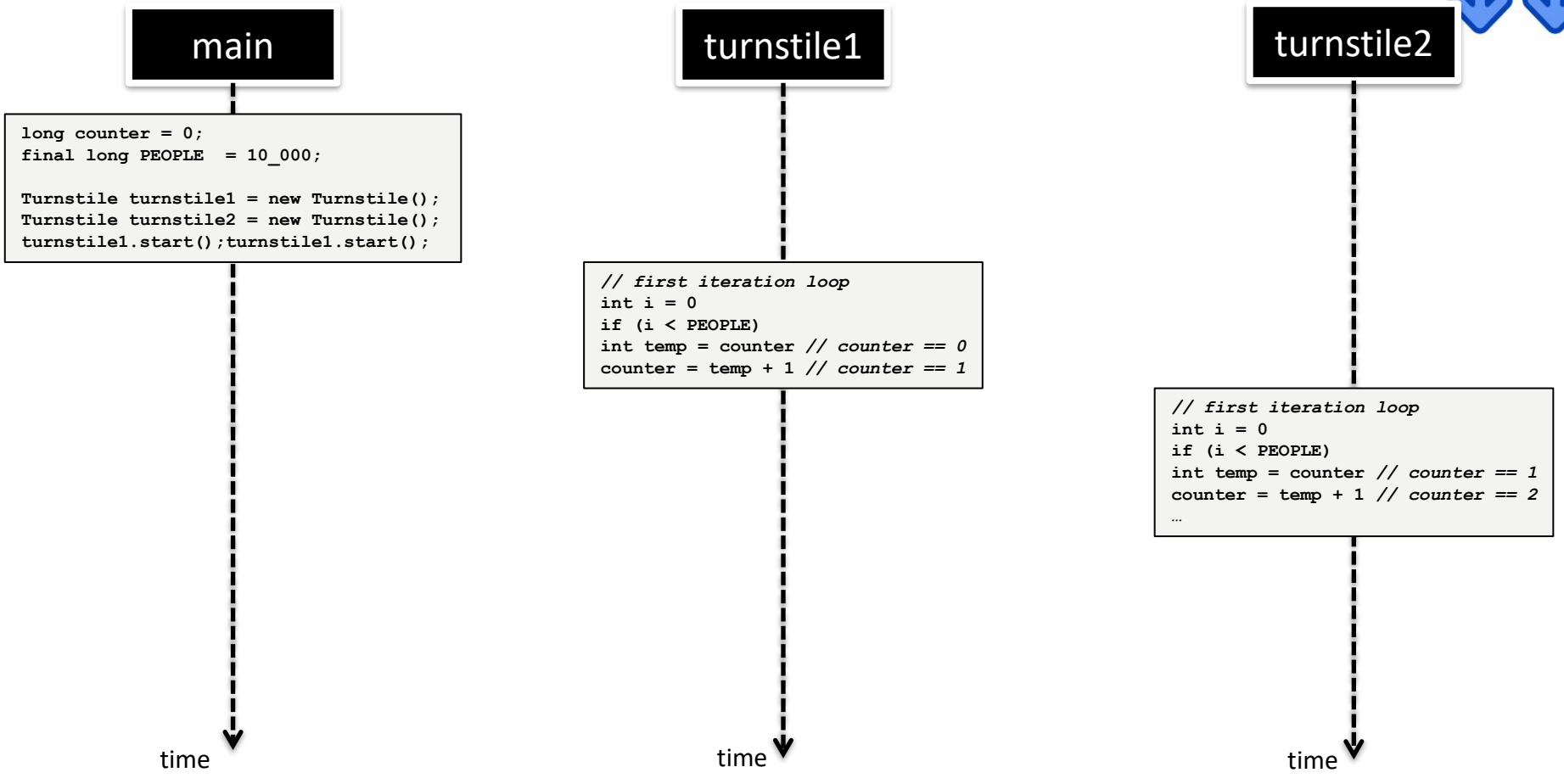
- Consider two threads t1 and t2 in the ready state; *t1(enabled)* and *t2(enabled)*
  1. *t1(running) -> t1(enabled) -> t1(running) -> t1(enabled) -> …*
  2. *t2(running) -> t2(enabled) -> t2(running) -> t2(enabled) -> …*
  3. *t1(running) -> t1(enabled) -> t2(running) -> t2(enabled) -> …*
  4. Infinitely many different executions!

Interleaving

- The statements in a thread are executed when the thread is in its “running” state

- An *interleaving* is a possible sequence of operations for a concurrent program
  - Note this: a sequence of operations for a concurrent program, not for a thread. Concurrent programs are composed by 2 or more threads.

Interleaving – Example I



Interleaving – Example II

main

```
long counter = 0;
final long PEOPLE  = 10_000;

Turnstile turnstile1 = new Turnstile();
Turnstile turnstile2 = new Turnstile();
turnstile1.start();turnstile1.start();
```

time

turnstile1

```
// first iteration loop
int i = 0
if (i < PEOPLE)
int temp = counter // counter == 0
```

```
counter = temp + 1 // counter == 1
```

time

turnstile2

```
// first iteration loop
int i = 0
if (i < PEOPLE)
int temp = counter // counter==0
```

```
counter = temp + 1 // counter == 1
…
```

time

Interleaving – Example II

main

```
long counter = 0;
final long PEOPLE  = 10_000;

Turnstile turnstile1 = new Turnstile();
Turnstile turnstile2 = new Turnstile();
turnstile1.start();turnstile1.start();
```

There are as many interleavings as possible ways to order the operations in the program

turnstile1

```
// first iteration loop
int i = 0
if (i < PEOPLE)
int temp = counter // counter == 0
```

```
counter = temp + 1 // counter == 1
```

time

turnstile2

```
// first iteration loop
int i = 0
if (i < PEOPLE)
int temp = counter // counter==0
```

```
counter = temp + 1 // counter == 1
…
```

time

Interleaving syntax

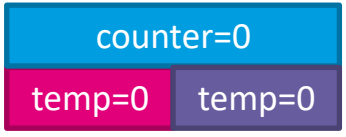
- The drawings above are not suitable for thinking about possible interleavings
- When asked to provide an interleaving, use the following syntax

```
<thread>(<step>), <thread>(<step>), …
```

Interleaving – Example I (textual)

Given the initial memory state on the right, provide an interleaving such that after two threads t1, t2 execute the program on the right the value of counter==2

Memory



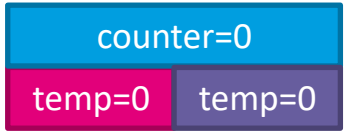
Program

```
public void run() {
  int temp = counter;  // (1)
  counter = temp + 1;  // (2)
}
```

Interleaving – Example I (textual)

Given the initial memory state on the right, provide an interleaving such that after two threads t1, t2 execute the program on the right the value of counter==2

Memory



Program

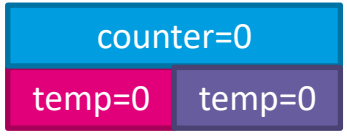
```
public void run() {
  int temp = counter;  // (1)
  counter = temp + 1;  // (2)
}
```

t1(1), t1(2), t2(1), t2(2)

Interleaving – Example I (textual)

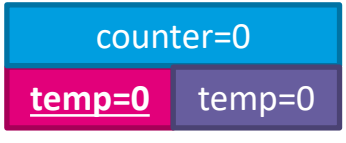
Given the initial memory state on the right, provide an interleaving such that after two threads t1, t2 execute the program on the right the value of counter==2

Memory



Program

```
public void run() {
  int temp = counter;  // (1)
  counter = temp + 1;  // (2)
}
```

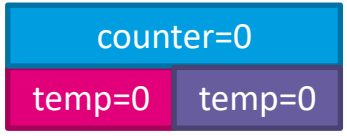


t1(1), t1(2), t2(1), t2(2)

Interleaving – Example I (textual)

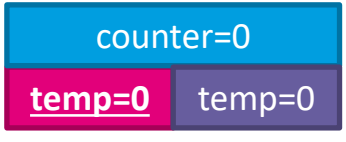
Given the initial memory state on the right, provide an interleaving such that after two threads t1, t2 execute the program on the right the value of counter==2

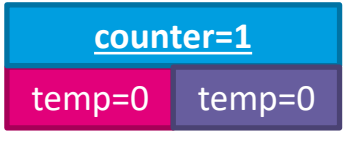
Memory



Program

```
public void run() {
  int temp = counter;  // (1)
  counter = temp + 1;  // (2)
}
```



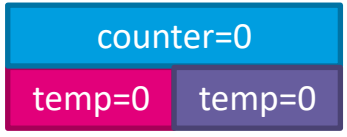


t1(1), t1(2), t2(1), t2(2)

Interleaving – Example I (textual)

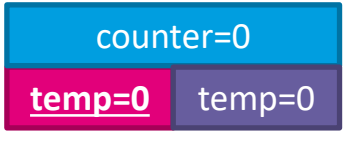
Given the initial memory state on the right, provide an interleaving such that after two threads t1, t2 execute the program on the right the value of counter==2

Memory

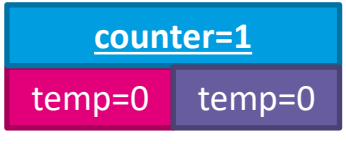


Program

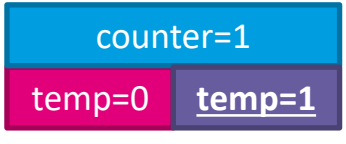
```
public void run() {
  int temp = counter;  // (1)
  counter = temp + 1;  // (2)
}
```



t1(1),



t1(2),



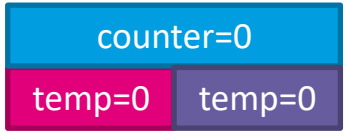
t2(1),

t2(2)

Interleaving – Example I (textual)

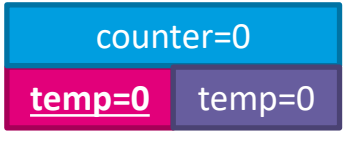
Given the initial memory state on the right, provide an interleaving such that after two threads t1, t2 execute the program on the right the value of counter==2

Memory

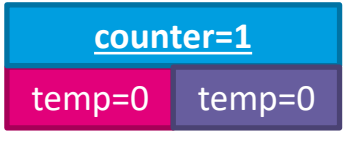


Program

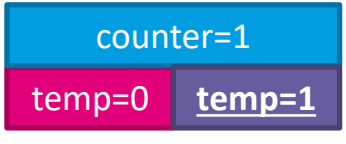
```
public void run() {
  int temp = counter;  // (1)
  counter = temp + 1;  // (2)
}
```



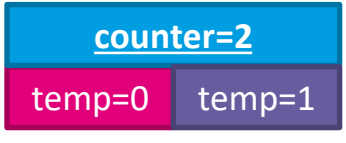
t1(1),



t1(2),



t2(1),



t2(2)

Race Conditions

- *A **race condition** occurs when the result of the computation depends on the interleavings of the operations*

Data Races

- *A **data race** occurs when two concurrent threads:*
  - *Access a shared memory location*
  - *At least one access is a write*

Race Conditions vs Data Races

Not all <u>race conditions</u> are <u>data races</u>

- Threads may not access shared memory
- Threads may not write on shared memory

Not all <u>data races</u> result in <u>race conditions</u>

- The result of the program may not change based on the writes of threads

Threads in Java – Example II

- What was is the problem in the previous program?
  - The statement **`counter++`** is not atomic
  - Some interleavings result in threads reading stale (outdated) data
  - Consequently, the program has race conditions that result in incorrect outputs
- In what follows, we will see how to tackle this type of problems

Critical sections

- A *critical section* is a part of the concurrent program where concurrent access must be restricted
  - Useful to avoid race conditions in concurrent programs

```
public class Turnstile extends Thread {
   public void run() {
        for (int i = 0; i < PEOPLE; i++) {
            // start critical section
            int temp = counter;
            counter = temp + 1;
            // end critical section
        }
   }
}
```

Critical sections

- A *critical section* is a part of the concurrent program where concurrent access must be restricted
  - Useful to avoid race conditions in concurrent programs

```
public class Turnstile extends Thread {
   public void run() {
        for (int i = 0; i < PEOPLE; i++) {
            // start critical section
            int temp = counter;
            counter = temp + 1;
            // end critical section
        }
   }
}
```

Critical sections should cover the parts of the code handling shared memory

Critical sections

- A *critical section* is a part of the concurrent program
  concurrent access must be restricted
  race conditions in concurrent progr

menti.com
6643 7213

Shouldn't the critical section start before the **`for`**?

```
nstile extends Thread {
    public void run() {
        for (int i = 0; i < PEOPLE; i++) {
            // start critical section
            int temp = counter;
            counter = temp + 1;
            // end critical section
        }
    }
}
```

Critical sections should cover the parts of the code handling shared memory

Mutual Exclusion

- The *mutual exclusion* property states that
  - *Two or more threads cannot execute their critical section at the same time*

- Mutual exclusion was first formulated by EW Dijkstra (see optional readings)
  - He devised a protocol to ensure mutual exclusion (*solving the mutual exclusion problem*)
  - He laid down the properties for a satisfactory solution to ensuring mutual exclusion

Mutual Exclusion

- An ideal solution to the mutual exclusion problem must ensure the following properties:
  - Mutual exclusion: at most one thread executes the critical section at the same time
  - Absence of *deadlock*: threads eventually exit the critical section allowing other threads to enter
  - Absence of *starvation*: if a thread is ready to enter the critical section, it must eventually do so

Mutual Exclusion

- An ideal solution to the mutual exclusion problem must ensure the following properties:
  - Mutual exclusion: at most one thread executes the critical section at the same time
  - Absence of *deadlock*: threads eventually exit the critical section allowing other threads to enter
  - Absence of *starvation*: if a thread is ready to enter the critical section, it must eventually do so

In practice, we will see that it is not always possible to achieve absence of starvation

Mutual Exclusion | Java `Locks`

- In Java, mutual exclusion can be achieved using the **`Lock`** interface in the **`java.util.concurrent.locks`** package
  - **`lock()`**
    - Acquires the lock if available, otherwise the thread executing lock() blocks
    - It is blocking
  - **`unlock()`**
    - If there are waiting theads, it signals one of them. Otherwise, it releases the lock. In either case, the thread executing unlock() continues execution
    - It is not blocking
- These are *synchronization operations*
  - They established an execution order among the operations of different threads

Java **Locks** | Critical sections

- Simple protocol: call **`lock()`** before entering the critical section, and **`unlock()`** after exiting
- *Each critical section must have a lock associated to it, but many critical sections may use the same lock.*
- Simplified, see **`CounterThreadsLock.java`**

```
Lock l = new Lock();

public class Turnstile extends Thread {
   public void run() {
        for (int i = 0; i < PEOPLE; i++) {
            l.lock()          // start critical section
            int temp = counter;
            counter = temp + 1;
            l.unlock()        // end critical section
        }
   }
}
```

Java `Locks` | Critical sections

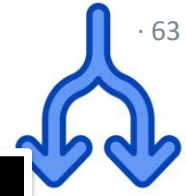
- Simple protocol: call **`lock()`** before entering the critical section, and **`unlock()`** after exiting
- *Each critical section must have a lock associated to it, but critical sections may use the same lock.*
- Simplified, see **`CounterThreadsLock.java`**

```
Lock l = new Lock();

public class Turnstile extends Threa
   public void run() {
        for (int i = 0; i < PEOPLE; i
            l.lock()          // start
            int temp = counter;
            counter = temp + 1;
            l.unlock()        // end critical section
        }
   }
}
```

- For now, we do not focus on the implementation of **`lock()`**/**`unlock()`**, but in their use to solve concurrency problems.
- In week 6 we will see how to implement several kinds of locks

Java `Locks` | Interleavings



main

```
long counter = 0;
final long PEOPLE  = 10_000;
Lock l = new Lock()

Turnstile turnstile1 = new Turnstile();
Turnstile turnstile2 = new Turnstile();
turnstile1.start();turnstile1.start();
```

turnstile1

```
// first iteration loop
int i = 0
```

```
if (i < PEOPLE)
```

```
l.lock() // begin critical section
int temp = counter // counter == 0
counter = temp + 1 // counter == 1
l.unlock() // end critical section
```

turnstile2

```
// first iteration loop
int i = 0
```

```
if (i < PEOPLE)
```

```
l.lock() // begin critical section
int temp = counter // counter == 1
counter = temp + 1 // counter == 2
l.unlock() // end critical section
```

Operations in the critical section are always executed sequentially by the same thread

Java `Locks` | Interleavings

- Interleaving leading to race condition, for two threads executing concurrently the program on the right

…,`t1(2)`,…,`t2(2)`,…,`t1(3)`,…,`t2(3)`,…

```
public void run() {
  l.lock();              // (1)
  int temp = counter;    // (2)
  counter = temp + 1;    // (3)
  l.unlock();            // (4)
}
```

Is this a valid interleaving?

Java `Locks` | Interleavings

- Interleaving leading to race condition, for two threads executing concurrently the program on the right

```
…,t1(2),…,t2(2),…,t1(3),…,t2(3),…
```

We can precisely show that the interleaving is not possible

```
public void run() {
  l.lock();              // (1)
  int temp = counter;    // (2)
  counter = temp + 1;    // (3)
  l.unlock();            // (4)
}
```

Java `Locks` | Interleavings

- Interleaving leading to race condition, for two threads executing concurrently the program on the right

```
public void run() {
  l.lock();              // (1)
  int temp = counter;    // (2)
  counter = temp + 1;    // (3)
  l.unlock();            // (4)
}
```

`…,t1(2),…,t2(2),…,t1(3),…,t2(3),…`

We can precisely show that the interleaving is not possible

`…,t1(2),…,t2(2),…,t1(3),…,t2(3),…` **[Assume]**

Java `Locks` | Interleavings

- Interleaving leading to race condition, for two threads executing concurrently the program on the right

```
public void run() {
  l.lock();              // (1)
  int temp = counter;    // (2)
  counter = temp + 1;    // (3)
  l.unlock();            // (4)
}
```

`…,t1(2),…,t2(2),…,t1(3),…,t2(3),…`

We can precisely show that the interleaving is not possible

`…,t1(2),…,t2(2),…,t1(3),…,t2(3),…` **[Assume]**

`…,t1(2),…,t1(4),…,t2(1),…,t2(2),t1(3),t2(3),…` **[t2(1) must be executed before t2(2)]**

Java `Locks` | Interleavings

- Interleaving leading to race condition, for two threads executing concurrently the program on the right

```
public void run() {
  l.lock();              // (1)
  int temp = counter;    // (2)
  counter = temp + 1;    // (3)
  l.unlock();            // (4)
}
```

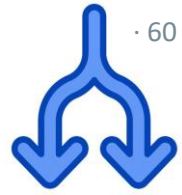
`…,t1(2),…,t2(2),…,t1(3),…,t2(3),…`

We can precisely show that the interleaving is not possible

`…,t1(2),…,t2(2),…,t1(3),…,t2(3),…` `[Assume]`

`…,t1(2),…,t1(4),…,t2(1),…,t2(2),t1(3),t2(3),…` `[t2(1) must be executed before t2(2)]` `[t1(4) must be executed before t2(1)]`

Java `Locks` | Interleavings



- Interleaving leading to race condition, for two threads executing concurrently the program on the right

```
…,t1(2),…,t2(2),…,t1(3),…,t2(3),…
```

We can precisely show that the interleaving is not possible

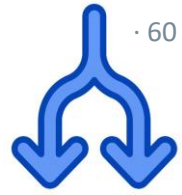
```
public void run() {
  l.lock();              // (1)
  int temp = counter;    // (2)
  counter = temp + 1;    // (3)
  l.unlock();            // (4)
}
```

```
…,t1(2),…,t2(2),…,t1(3),…,t2(3),…                          [Assume]

…,t1(2),…,t1(4),…,t2(1),…,t2(2),t1(3),t2(3),…             [t2(1) must be executed before t2(2)]
                                                           [t1(4) must be executed before t2(1)]

…,t1(2),…,t1(3),…,t1(4),…,t2(1),…,t2(2),t1(3),t2(3),…     [t1(3) must be executed before t1(4)]
                                                           [t1(2) must be executed before t1(3)]
```

Java `Locks` | Interleavings



- Interleaving leading to race condition, for two threads executing concurrently the program on the right

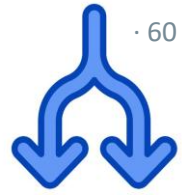
```
public void run() {
  l.lock();              // (1)
  int temp = counter;    // (2)
  counter = temp + 1;    // (3)
  l.unlock();            // (4)
}
```

…,t1(2),…,t2(2),…,t1(3),…,t2(3),…

We can precisely show that the interleaving is not possible

…,t1(2),…,t2(2),…,t1(3),…,t2(3),…	[Assume]
…,t1(2),…,t1(4),…,t2(1),…,t2(2),t1(3),t2(3),…	[t2(1) must be executed before t2(2)] [t1(4) must be executed before t2(1)]
…,t1(2),…,t1(3),…,t1(4),…,t2(1),…,t2(2),t1(3),t2(3),…	[t1(3) must be executed before t1(4)] [t1(2) must be executed before t1(3)] [Contradiction]

Java `Locks` | Interleavings



- Interleaving leading to race condition, for two threads executing concurrently the program on the right

```
…,t1(2),…,t2(2),…,t1(3),…,t2(3),…
```

```
public void run() {
  l.lock();              // (1)
  int temp = counter;    // (2)
  counter = temp + 1;    // (3)
  l.unlock();
}
```

We can precisely show that the interleaving is not possible

Memory models determine valid executions of a concurrent program, we discuss the Java memory model in week 3

```
…,t1(2),…,t2(2),…,t1(3),…,t2(3),…                                  [Assume]

…,t1(2),…,t1(4),…,t2(1),…,t2(2),t1(3),t2(3),…                      [t2(1) must be executed before t2(2)]
                                                                    [t1(4) must be executed before t2(1)]

…,t1(2),…,t1(3),…,t1(4),…,t2(1),…,t2(2),t1(3),t2(3),…              [t1(3) must be executed before t1(4)]
                                                                    [t1(2) must be executed before t1(3)]
                                                                    [Contradiction]
```

Deadlocks

- This solution to the turnstile problem ensures mutual exclusion **but does it ensure absence of deadlock?**

Absence of *deadlock*: threads eventually exit the critical section allowing other threads to enter

```
Lock l = new Lock();

public class Turnstile extends Thread {
   public void run() {
        for (int i = 0; i < PEOPLE; i++) {
            l.lock()          // start critical section
            int temp = counter;
            counter = temp + 1;
            l.unlock()        // end critical section
        }
   }
}
```

(Naïve) examples of deadlocks

- Locking twice within the thread

```
for (int i = 0; i < PEOPLE; i++) {
     l.lock()
     int temp = counter;
     counter = temp + 1;
     l.lock()  // blocks forever
}
```

Thread 1

This is not unrealistic. For instance, see these bug reports in the Linux kernel

[1] https://github.com/torvalds/linux/commit/e1db4ce
[2] https://github.com/torvalds/linux/commit/16da4b17
[3] https://github.com/torvalds/linux/commit/cdc9718d5e590d6905361800b938b93f2b66818e

- Exception during execution

```
for (int i = 0; i < PEOPLE; i++) {
     l2.lock()
     int temp = counter;
     throw new Exception();
     // If exception handling doesn't
     unlock, blocks forever

}
```

Thread 1

```
for (int i = 0; i < PEOPLE; i++) {
     l2.lock() // blocks forever
     …
}
```

Thread 2

Java `Lock` Idiom

- When using Java locks, it is recommended to use the idiom on the right
- This prevents deadlocks problems if the thread finishes unexpectedly due to exception
  - Solutions not following this idiom cannot be deemed as free from deadlocks (note for assignments)
- We use this idiom in **`CounterThreadLocks.java`**

```
Lock l = new Lock();

l.lock()
try {
   // critical section code
} finally {
   l.unlock()
}
```

Java ReentratLock

- Java **Lock** is an interface, so it cannot be used as we showed in the examples today
- We use an implementation of the **Lock** interface, namely **ReentrantLock**
  - Reentrant locks act like a regular lock, except that they allow locks to be locked more than once by the same thread

```
Lock l = new ReentrantLock();

for (int i = 0; i < PEOPLE; i++) {
    l.lock()
    int temp = counter;
    counter = temp + 1;
    l.lock();     // it doesn't block
    l.unlock();   // still holds the lock
    l.unlock();   // now the lock is free
}
```